

Cartesian control in SMC

Marko Guberina

September 24, 2025

0.1 Cartesian control

There is a generic point-to-point a generic trajectory following ControlLoopTemplate. The point-to-point template computes the error vector as

$$e = \log_6 \left((T_w^e)^{-1} \cdot T_w^{\text{goal}} \right) \quad (1)$$

which means e is a twist (6D velocity), i.e. $e \in se(3)$. It then calls on an `ik_solver` which is specified with an argument. The available inverse kinematics solvers are listed below they all take the error twist and the Jacobian as arguments, and return their joint velocity solution.

The only thing that the path-following does differently is how the

0.1.1 Point to point cartesian control

Point to point cartesian control

1. compute $SE(3)$ error $E = \log_6 \left((T_w^e)^{-1} \cdot T_w^{\text{goal}} \right) = T_e^{\text{goal}}$, i.e. the transformation from end-effector frame to goal frame
2. map to $se(3)$ via the log map: $V_{\text{error}} = \log_6(T_e^{\text{goal}})$.
3. compute joint velocity v such that the end-effector moves with twist V_{error} with an `ik_solver` with $v_{\text{cmd}} = \text{ik_solver}(J, v_{\text{error}})$

Short note on the log operation The twist V_{error} corresponds to the velocity such that moving from frame $\{e\}$, with twist V_e , for one unit of time, leads to the $\{g\}$ frame. Said another way, if $\{V_e\}$ is the body velocity of the end-effector frame for one unit of time, it gets to the goal frame. The reason this works has to do with Lie algebra. Detailed overview can be found in ex. Frank C. Park, Kevin Lynch - Modern Robotics or Murray, R.M., Li, Z. and Sastry, S.S., 2017. A mathematical introduction to robotic manipulation. CRC press, or on [this excellent blog post](#).

0.1.2 Inverse kinematics solvers

There are of course the classics:

- Jacobian pseudoinverse: $v = J^\dagger V_e = J^T(JJ^T + \lambda I)^{-1}V_e$
- Jacobian transpose: $v = J^T V_e$

For larger movement, these methods can result in wonky trajectories as the error twist V_e is large, resulting in large joint velocity commands v . Those in turn get saturated, leading to the end-effector not going in a straight line, but something else. This could be solved by scaling the error twist V_e to a smaller number, but this is not an ideal solution because the maximum end-effector twist in a particular direction depends on the configuration (which could be taken into account manually, but there are better methods out there).

QP formulation The inverse kinematics problem can be posed as an optimization problem:

$$\min_v \frac{1}{2} \|J\Delta q + wV\| \quad (2)$$

$$\text{s.t. } v_{\min} \leq v \leq v_{\max} \quad (3)$$

The joint displacement Δq can be regarded as joint velocity v if the integration step is small enough (it is, i.e. we make sure that we have high enough control frequency). From now on we write v . If there were no additional objectives, and no any constraints whatsoever, the linear least-squares solution to this problem would be the Jacobian pseudoinverse method.

This can be transcribed as a generic quadratic program (QP), which looks like

$$\min_x \frac{1}{2} x^T P x + q^T x \quad (4)$$

$$\text{s.t. } Gx \leq h \quad (5)$$

$$Ax = b \quad (6)$$

In particular, the transcription goes as follows:

$$\|Jv - KV\|^2 = (Jv - KV)^T (Jv - KV) \quad (7)$$

$$= v^T J^T J v - KV^T J v - Kv J^T V + \underbrace{K^2 V^T V}_{\text{does not depend on } v} \quad (8)$$

$$\implies v^T \underbrace{(J^T J)}_P v - \underbrace{2KV^T J}_r v \quad (9)$$

Note that the same Tikhonov regularization (also known as Levenberg-Marquardt damping in this context) should be added to the P as it was in the pseudoinverse, i.e. $P = J^T J + \lambda I$. Crucially, we can add additional tasks and constraints

to the QP formulation, as long as the costs are quadratic, and the constraints are linear. Since we are solving this problem at each control time step, this is actually quite powerful as we can use linearised nonlinear costs/constraints and they will work fine.

In particular, let a cost be denoted as $e(q)$, and it's Jacobian by $J(q)$. This will again result in a cost of form $\|J_i v - K_i e_i\|^2$. You then work out the same steps as above for the frame placement cost, getting (P_i, r_i) . Then you can simply put $P = \sum_i P_i, r = \sum_i r_i$, which is still quadratic. This emulates behaviour of HQP.

Finding an appropriate secondary task, and doing that computation is a part of the TODO. Since the biggest issue on YuMi is the elbows hitting the torso, going for $e = q_{\text{comfy_configuration}} - q$ is probably the way to go. The alternatives are manipulability maximization or self-collision avoidance. Manipulability maximization could also be added as a ternary goal, where the manipulability is defined for the whole of the robot. Looking at [pink](#) and [it's documentation](#) is a good place to start. **Note** that pink isn't integrated into SMC simply because it was annoying to match it's structure to SMC's, but it can be done. A starting point for that approach would be to look into `examples/cartesian_space/yumi_self_collision.py`.

Solution approach We need the QP formulation of inverse kinematics, (or something which allows for additional objectives and constraints), because the pseudoinverse solver causes the configuration of the robot goes to shit (breaks the robot) during Cartesian path-following (and thus cart-pulling as well).

One way to implement the QP formulation is to shove the whole of pink setup into an `ik_solver` construction. Then it can be applied to any `controlLoop` without any additional code. That requires creating a big struct for all pink's variables, which is then partialled to comply with `ik_solver`'s API. The API could also be changed, just note that you then need to refactor this throughout the codebase.

The way adding tasks is handled in pink is great, and there is no point in rewriting that. We want this because it would make it super easy to swap different objectives, i.e. we wouldn't need to a new ik solver for every thing to try (or write the logic equivalent to pink to make one of the customizable).

Alternatively, we can manually construct the QP, and also partial in whatever additional variables we need. The drawback is that this will be more verbose and annoying, but the upside is that the math of the QP will be completely transparent.

The code for all inverse kinematics solvers is in `python/smc/control/cartesian_space/ik_solvers.py`.

Available QP solvers:

- solvers supported by [quadprog](#) (we use what the default one is, but you can install and select any one supported by quadprog)
- [proxsuite](#) - cool because it has warmstarting (among other things)

Secondary goals

- posture reference tasks with $e(q) = q_{\text{comfy}} - q \rightarrow \text{DONE}$
ik_solver is called `QPWithPosture`. It does not take mobile poses positions into account of course.
- velocity limit constraints $\rightarrow \text{DONE}$
- TODO: joint limit constraints
- TODO IF ABOVE ISN'T ENOUGH FOR GOOD POSTURE: put in the derivative of manipulability w.r.t. q .
- TODO IF SELF-COLLISIONS ARE COMMON: adding collision avoidance via control barrier functions. Regarding what to avoid, there are 2 options:
 1. make spheres around elbows and end-effectors. Use `BodySphericalBarrier` from pink for this. Inputs are frames that can't be avoided and the radius of the sphere around said frames. Look at `pink/examples/barriers/yumi_self_collision.py` for details. There's an SMC-ed version of this in `examples/cartesian_control/yumi_self_collision.py`
 2. make ellipses around every link. Use `SelfCollisionBarrier` from pink for this. Look at `pink/examples/barriers/kukas_self_collision.py` for details.

How to run The main file for SingleArmInterface point to point Cartesian control is `example/cartesian_space/клик_point_to_point.py`, and the one for DualArmInterface is `example/cartesian_space/dual_arm_клик.py`. The relevant arguments can be found simple by adding `-help` when running the example, but the most important ones is `-ik-solver [solver]` to select a solver.

0.1.3 Path following Cartesian control

It's sad how stupidly Cartesian path following is implemented.

It works on $SE(3)$ paths, i.e. there are no pre-determined timing laws. The controller assumes that the first point of the path coincides with the current pose of the thing we're planning for. The responsibility to ensure this is on the path-planner (the current pose is accessible to it). While one might argue that there should be a handling routine for when this isn't the case, because the path planner is slower than the controller, we simply don't have this problem at the moment, so we haven't solved it.

The path-following controller computes the positional error twist the same way as the point-to-point version, from the end-effector to the second point of the path. Does the same between the second and third point of the

path, and treats that as a reference velocity. This is obviously stupid because the quality of it depends on spacing of the path points, and there are circumstances where this causes performance issues. [In Marko's opinion] It is the responsibility of the controller to smooth out the given path (from the web programming mantra "be loose with what you accept, be rigid in what you provide" (yes this is in opposition to assuming the first path point coincides with the end-effector, but again, that wasn't a problem (yet))).

[ABHISHEK TODO]: This is to be fixed with path interpolation. The algorithm should then act as follows:

Cartesian path following controller

1. (optional) find the current path point (where you are on the path)
2. determine how many points in advance should be interpolated (as few as possible)
3. interpolate those points to create a smooth reference (poly, Bezier or w/e)
4. (possibly necessary) low-pass filter update this with the previous path in the case of following from a path planner, to have smoother motion if/when the plan changes by a lot
5. impose some timing law on the path as then makes positional control makes sense. The timing law choice should be connected to the heuristic on number/length of interpolated path points .
6. compute desired velocity (most likely feedback position error, feedforward velocity, but this probably depends on how the interpolation is done??? idk)
7. apply an inverse kinematics solver to find joint velocities to achieve desired velocity

More details on this are given below.

The existing code can be found in

`python/smc/control/cartesian_space/cartesian_space_trajectory_following.py`.

[ABHISHEK TODO]: path interpolation This is the key thing to do to make sure Cartesian path following works. The first thing to do is determine the number of points to be interpolated between. There is no benefit in interpolating the whole path as we're not following the whole path. We want a simple curve to make the computation quick, as it needs to happen in real-time in conjunction with the real-time path planner (anything reasonable should be fine). **Note** that linear interpolation (without splines) isn't good enough as it results in jagged motion on the "edges".

One option is to form a Bezier curve, or a polynomial interpolation on the first few points and call it a day. A smarter version would be to do the same, but with least-squares fit on a larger number of points, the amount of which is determined by some heuristic like their arclength being at least something.

Deciding on the exact strategy is a part of the TODO.

Short note on 2D to 6D reference Often we get a 2D path to follow, which is of course underdefined. This is most common when drawing a path for the purposes of testing a path-following controller. The path is made to be 3D by specifying a height h (this is an argument), giving the translation $p = [x_i \ y_i \ h]$. Of course, we could impose an arbitrary orientation along the path, and interpolate the $SO(3)$ rotations to make it smooth. However, this isn't implemented as it (so far) isn't useful for cart-pulling nor navigation.

Instead, a Frenet frame is constructed on the linear path to give it orientation. First, we make the x-axis be the tangent of the path. The angle of the x-axis of the frame as seen in the x-y plane of the world frame $\{w\}$ is determined with

$$\theta[i] = \text{atan2} \left(\frac{\phi_y[i+1] - \phi_y[i]}{\phi_x[i+1] - \phi_x[i]} \right) \quad (10)$$

and we turn this into a rotation matrix with $R_{\theta_i} \text{rpyToMatrix}(0, 0, \theta_i)$. Next, we specify the rotation about the x-axis (this is an argument), and get $R_x = \text{rpyToMatrix}(\text{rot_x}, 0, 0)$. In cart-pulling, we want the z axis to be perpendicular to the y-axis. The y-axis completes the right-handed frame, meaning $R = R_{\theta} \cdot R_x$. Finally, the resulting $SE(3)$ path is then

$$P_i = \begin{bmatrix} R_i & p_i \\ 0 & 1 \end{bmatrix} \quad (11)$$

How to run The main file for SingleArmInterface path following Cartesian control is `example/cartesian_space/clik_path_following.py`, and the one for DualArmInterface is `example/cartesian_space/dual_arm_clik_path_following.py`. The relevant arguments can be found simple by adding `-help` when running the example, but the most important ones is `-ik-solver [solver]` to select a solver. Additionally, there is a `-planner` option (on by default). This will spawn the real-time path generation in a new process, and create a predefined map.